Research Foundation and Computational Work Plan

Week 1 Foundation and Week 2 Research Method Implementation

Research project: Comparative Study of Explainable AI Models for Pneumonia Detection from Chest X-rays

Document purpose: describe the research question, real-world research data, computational approach, initial implementation, reproducibility plan, and a practical schedule for Week 2.

1. Research Problem and Purpose

Pneumonia detection from chest X-ray images is a classification task in which a model assigns an image to a category such as NORMAL or PNEUMONIA. A model's score alone does not show which image regions influenced its prediction. This project will compare two image-classification architectures and examine their visual explanations. The work is an academic computational experiment, not a medical device or a tool for diagnosing patients.

2. Primary Research Question

How do ResNet-50 and DenseNet-121 differ in pneumonia-detection performance, and how do their Grad-CAM explanations compare on the same chest X-ray images?

3. Working Hypothesis

The two model architectures may produce different classification results and may highlight different image regions when Grad-CAM is applied to the same chest X-rays. The study will not assume in advance that one model is better. The hypothesis will be assessed using consistent test images, the same evaluation metrics, and a documented explanation procedure.

4. Evidence Needed to Answer the Question

- Dataset evidence: image counts for each class and each available split, image formats, file sizes, unreadable-image counts, and any missing expected folders.
- Classification evidence: accuracy, precision, recall, F1-score, ROC-AUC when valid probability scores and both classes are available, and confusion matrices for each model.
- Comparison evidence: model results calculated on the same held-out test examples using the same label mapping and evaluation procedure.
- Explainability evidence: Grad-CAM overlays for selected test images, including correct predictions, incorrect predictions, and cases where the models disagree.
- Reproducibility evidence: dependency versions, random seeds where applicable, model/checkpoint details, preprocessing choices, execution commands, and saved result files.
- Interpretive evidence: error analysis and limitations, including class imbalance, dataset provenance, potential data leakage, and the fact that Grad-CAM is not proof of medically correct reasoning.

5. Research Data Source

The planned dataset is the public Chest X-Ray Images (Pneumonia) collection hosted on Kaggle. It contains real chest X-ray images labelled NORMAL or PNEUMONIA and is commonly distributed with train, validation, and test folders. Dataset page: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia

This is a real, existing image dataset, not synthetic data and not a live real-time feed. It is suitable for a controlled baseline experiment because both models can be evaluated on the same images. Before analysis, review the dataset page, terms of use, documentation, and provenance. The exact image counts and folder structure must be confirmed locally by running the inspection script; this document does not claim that the data has already been downloaded or verified.

Data handling: download the dataset from its source and keep the image files outside GitHub. Do not upload identifiable or restricted data. Commit code, documentation, and appropriate small summary outputs only. Record the download date and the source/version information in your experiment notes.

6. Computational Environment and Repository Structure

Use Python 3.11 in a virtual environment. The initial data inspection can run with the Python standard library. The initial dependency file includes NumPy, pandas, Pillow, Matplotlib, and scikit-learn. For the model experiment, add PyTorch and torchvision; use a Grad-CAM implementation such as the pytorch-grad-cam package after confirming compatibility with the installed PyTorch version. A GPU-enabled environment such as Google Colab may be used if local hardware is insufficient, but record the environment and versions used.

```
repository-root/
├── docs/
│   └── step_2/
│       └── week_1/
│           └── research_foundation.pdf (Week 1 submission copy, if required)
├── src/
│   ├── __init__.py
│   └── data/
│       ├── __init__.py
│       └── inspect_dataset.py
├── data/
│   └── chest_xray/          (download locally; do not commit)
├── outputs/
│   └── week_1/              (generated local reports; generally ignored by Git)
```

├── requirements.txt

├── .gitignore

└── README.md

7. Initial Code and What It Does

The Week 1 script src/data/inspect_dataset.py checks for the expected train, validation, and test folders and the NORMAL and PNEUMONIA class folders. It counts supported image files and creates an image_manifest.csv with the split, label, file path, extension, and file size. It also writes dataset_summary.txt and reports any missing expected directories. This is a meaningful first research operation because it verifies whether the data is available and provides an initial class/split inventory.

The script does not yet read image pixels to check corruption, train either model, calculate classification metrics, or create Grad-CAM maps. Those are planned next steps. Do not describe the first script as a completed model experiment.

8. Instructions to Run the Week 1 Code

1. Open a terminal at the repository root.

2. Create a virtual environment: python -m venv .venv

3. Activate it in Windows PowerShell: .venv\Scripts\Activate.ps1

   For Windows Command Prompt: .venv\Scripts\activate.bat

   For macOS/Linux: source .venv/bin/activate

4. Install dependencies: python -m pip install --upgrade pip

5. Install packages: pip install -r requirements.txt

6. Download and extract the dataset so data/chest_xray/train/NORMAL and data/chest_xray/train/PNEUMONIA exist, along with the corresponding validation/test folders if supplied.

7. Run: python src/data/inspect_dataset.py --data-root data/chest_xray --output-dir outputs/week_1

8. Review outputs/week_1/image_manifest.csv and outputs/week_1/dataset_summary.txt.

9. Initial Risks, Limitations, and Design Decisions

- The actual dataset structure and image counts must be confirmed by execution; they should not be guessed or copied from online descriptions.
- The dataset may be imbalanced across NORMAL and PNEUMONIA labels, which can make accuracy alone misleading. Report precision, recall, F1-score, and confusion matrices as well.
- The provided split folders do not automatically guarantee patient-level independence. Investigate dataset metadata and document whether patient-level overlap can be checked.
- Image files may be unreadable or inconsistent in size, format, or quality. Add image validation before model evaluation.
- Model comparison is only fair when both models use the same held-out test images, consistent labels, and clearly documented preprocessing.
- Grad-CAM highlights regions associated with a model's output; it does not establish causality, clinical validity, or the correctness of a diagnosis.
- Compute limits may affect training time. Begin with pretrained models and a small pilot run, then document any justified adjustments.

10. Week 2 Objective

Week 2 will move from data inventory to an initial controlled baseline experiment. The goal is to implement an executable image pipeline and obtain preliminary, reproducible measurements from at

least one model, while preparing a fair comparison with the second model. The experiment should produce real outputs from the dataset rather than hard-coded or manually invented results.

11. Week 2 Experimental Procedure

1. **Data validation:** Run the inventory script; add checks that each file can be opened and decoded. Save counts of valid and invalid images and investigate failures.
2. **Preprocessing:** Define one label mapping (NORMAL=0, PNEUMONIA=1), image size, normalization, and transformations. Apply the same evaluation preprocessing to both models.
3. **Baseline condition:** Use a pretrained ResNet-50 as the first baseline and replace its final classification layer for the two classes. Fine-tune only as compute permits. Record whether the backbone is frozen or trainable.
4. **Comparison condition:** Prepare DenseNet-121 using the same data splits, label mapping, evaluation preprocessing, and metric code. If time or compute is limited, run a pilot and clearly state which model has actually completed.
5. **Evaluation:** Use the validation split for model selection and the supplied test split for final pilot metrics. Do not tune hyperparameters on the test set. Calculate accuracy, precision, recall, F1-score, ROC-AUC where possible, and confusion matrix.
6. **Explanation analysis:** Generate Grad-CAM overlays for a small, explicitly selected sample of test images after the classification pipeline works. Include correct, incorrect, and model-disagreement cases where available.
7. **Recording:** Save predictions, metrics, plots, configuration, random seed, library versions, and execution notes in an outputs/ or results/ location. Do not commit the full image dataset.
8. **Validation checks:** Check that predicted labels and probabilities have the expected shape, metric calculations use the correct positive class, both classes are represented where ROC-AUC is calculated, and test examples are not used for tuning.

12. Week 2 Schedule (Seven-Day Plan)

Day	Tasks	Evidence / Deliverable
Day 1	Download data, inspect source terms and folder layout, run Week 1 script.	Dataset summary and manifest; notes on missing folders or class counts.
Day 2	Implement image-open/decode checks and save a list/count of invalid files.	Validation report and tests for missing or unreadable files.
Day 3	Implement preprocessing, label mapping, and reproducible train/validation/test data loaders.	Preprocessing code, configuration, and a small batch shape check.
Day 4	Set up PyTorch/torchvision and implement pretrained ResNet-50 baseline.	Executable baseline training/evaluation script and logged configuration.
Day 5	Run a pilot; calculate classification metrics and save predictions/confusion matrix.	Metrics CSV or JSON, confusion matrix, and run log with honest limitations.
Day 6	Set up DenseNet-121 with the same evaluation protocol; compare if compute permits.	Comparable metrics for both models, or documented pilot status if unfinished.
Day 7	Add initial Grad-CAM examples if feasible, write	Code in src/, updated dependencies, Week 2

	experiment notes, run tests, and push changes.	results/notes, and GitHub commit.

13. Week 2 Acceptance Checklist

- Research code is organized under src/ and can be run from documented commands.
- Dataset inventory and image validation run on the actual downloaded dataset.
- Preprocessing and split usage are documented and reproducible.
- At least one baseline model produces real predictions and measurable evaluation outputs.
- The reference condition and metrics are defined; a second model is implemented or its remaining work is clearly documented.
- Outputs are saved and not fabricated; test data is not used for model selection.
- Validation checks and known limitations are recorded.
- Changes are committed and pushed to GitHub, with documents under docs/step_2/ and code under src/.

14. Expected Outcome at the End of Week 2

The expected outcome is a functioning pilot research pipeline that loads the selected chest X-ray dataset, applies documented preprocessing, runs an initial baseline model, and records preliminary evaluation evidence. If compute time allows, the same protocol will be used to evaluate DenseNet-121 and generate initial Grad-CAM examples. Results may be incomplete at this stage. Any uncompleted experiment, technical issue, or change to the protocol should be stated clearly rather than replaced with assumed results.